Министерство образования Республики Беларусь
Учреждения образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №3

По дисциплине «Современные платформы программирования»

Выполнил:
Студент 3 курса
Группы ПО-12
Матюх И.М.
Проверил:
Кулик А. Д.

Брест 2026

Цель работы: приобрести навыки применения паттернов проектирования при решении практических задач с использованием языка Python.

Вариант: №12

Задание 1: Заводы по производству автомобилей. Реализовать возможность создавать автомобили различных типов на различных заводах.

Код программы:

```
from abc import ABC, abstractmethod
class Car(ABC):
    def __init__(self, model, engine, price):
        self.model = model
        self.engine = engine
        self.price = price
    @abstractmethod
    def get_description(self):
        pass

    def __str__(self):
        return f"{self.model} ({self.engine}) - {self.price} руб."

class Sedan(Car):
    def get_description(self):
        return f"Седан {self.model}: комфортный городской автомобиль с двигателем
{self.engine}"

    def drive_city(self):
        return f"Едем по городу на {self.model}. Плавно и экономично."

class SUV(Car):
    def get_description(self):
        return f"Внедорожник {self.model}: мощный автомобиль с двигателем {self.engine}
для бездорожья"

    def drive_offroad(self):
        return f"Едем по бездорожью на {self.model}. Полный привод включен!"
class Hatchback(Car):
    def get_description(self):
        return f"Хэтчбек {self.model}: компактный и маневренный автомобиль с двигателем
{self.engine}"

    def park_easily(self):
        return f"Паркуемся на {self.model}. Компактные размеры - легко!"
class CarFactory(ABC):

    @abstractmethod
    def create_sedan(self) -> Sedan:
        pass

    @abstractmethod
    def create_suv(self) -> SUV:
        pass

    @abstractmethod
    def create_hatchback(self) -> Hatchback:
        pass

    @abstractmethod
    def get_factory_name(self) -> str:
        pass

class VolkswagenFactory(CarFactory):

    def get_factory_name(self) -> str:
        return "Завод Volkswagen (Германия)"

    def create_sedan(self) -> Sedan:
```

```
        return Sedan("Volkswagen Jetta", "1.4 TSI (бензин)", 1800000)

    def create_suv(self) -> SUV:
        return SUV("Volkswagen Tiguan", "2.0 TDI (дизель)", 2800000)

    def create_hatchback(self) -> Hatchback:
        return Hatchback("Volkswagen Golf", "1.6 MPI (бензин)", 1600000)


class ToyotaFactory(CarFactory):

    def get_factory_name(self) -> str:
        return "Завод Toyota (Япония)"

    def create_sedan(self) -> Sedan:
        return Sedan("Toyota Camry", "3.5 V6 (бензин)", 2500000)

    def create_suv(self) -> SUV:
        return SUV("Toyota Land Cruiser Prado", "2.8 TD (дизель)", 4500000)

    def create_hatchback(self) -> Hatchback:
        return Hatchback("Toyota Corolla", "1.8 Hybrid", 2000000)


class TeslaFactory(CarFactory):

    def get_factory_name(self) -> str:
        return "Завод Tesla (США)"

    def create_sedan(self) -> Sedan:
        return Sedan("Tesla Model S", "электрический (1020 л.с.)", 8500000)

    def create_suv(self) -> SUV:
        return SUV("Tesla Model X", "электрический (1020 л.с.)", 9500000)

    def create_hatchback(self) -> Hatchback:
        return Hatchback("Tesla Model 3", "электрический (450 л.с.)", 5000000)
def get_factory_choice():
    print("\nДоступные заводы:")
    print("1. Volkswagen (Германия)")
    print("2. Toyota (Япония)")
    print("3. Tesla (США)")

    while True:
        try:
            choice = int(input("\nВыберите завод (1-3): "))
            if choice == 1:
                return VolkswagenFactory()
            elif choice == 2:
                return ToyotaFactory()
            elif choice == 3:
                return TeslaFactory()
            else:
                print("Ошибка: введите число от 1 до 3")
        except ValueError:
            print("Ошибка: введите число")
def get_car_type_choice():
    print("\nДоступные типы автомобилей:")
    print("1. Седан")
    print("2. Внедорожник (SUV)")
    print("3. Хэтчбек")

    while True:
        try:
            choice = int(input("\nВыберите тип автомобиля (1-3): "))
            if choice == 1:
                return "sedan"
            elif choice == 2:
                return "suv"
```

```
            elif choice == 3:
                return "hatchback"
            else:
                print("Ошибка: введите число от 1 до 3")
        except ValueError:
            print("Ошибка: введите число")
def order_car(factory: CarFactory, car_type: str):
    print(f"\n{'=' * 50}")
    print(f"ОФОРМЛЕНИЕ ЗАКАЗА НА {factory.get_factory_name()}")
    print("=" * 50)
    if car_type == "sedan":
        car = factory.create_sedan()
        print(f"\n✅ Вы выбрали: {car}")
        print(car.get_description())
        print(car.drive_city())
    elif car_type == "suv":
        car = factory.create_suv()
        print(f"\n✅ Вы выбрали: {car}")
        print(car.get_description())
        print(car.drive_offroad())
    elif car_type == "hatchback":
        car = factory.create_hatchback()
        print(f"\n✅ Вы выбрали: {car}")
        print(car.get_description())
        print(car.park_easily())
    print(f"\n💰 Итоговая стоимость: {car.price} руб.")
    buy = input("\nХотите купить этот автомобиль? (да/нет): ").lower()
    if buy in ["да", "yes", "у", "д"]:
        print("\n🎉 ПОЗДРАВЛЯЕМ С ПОКУПКОЙ!")
        print(f"Вы стали владельцем {car.model}!")
    else:
        print("\nЗаказ отменен.")

    return car
def main():

    print("=" * 70)
    print("ПРОГРАММА: ЗАВОДЫ ПО ПРОИЗВОДСТВУ АВТОМОБИЛЕЙ")
    print("=" * 70)
    print("\nДобро пожаловать в систему заказа автомобилей!")

    while True:

        factory = get_factory_choice()

        car_type = get_car_type_choice()

        order_car(factory, car_type)

        again = input("\nХотите сделать еще один заказ? (да/нет): ").lower()
        if again not in ["да", "yes", "у", "д"]:
            break

    print("\n" + "=" * 70)
    print("СПАСИБО ЗА ИСПОЛЬЗОВАНИЕ ПРОГРАММЫ!")
    print("=" * 70)

if __name__ == "__main__":
    main()
```

```
✅ Вы выбрали: Volkswagen Jetta (1.4 TSI (бензин)) - 1800000 руб.
Седан Volkswagen Jetta: комфортный городской автомобиль с двигателем 1.4 TSI (бензин)
Едем по городу на Volkswagen Jetta. Плавно и экономично.

💰 Итоговая стоимость: 1800000 руб.
```

Задание 2: Проект «Универсальная электронная карта». В проекте должна быть реализована универсальная электронная карта, в которой есть функции паспорта, страхового полиса, банковской карты и т. д.

Код программы:

```
from abc import ABC, abstractmethod
from datetime import datetime, timedelta


class PassportSystem:

    def __init__(self, full_name, birth_date, passport_number, issue_date, issued_by):
        self.full_name = full_name
        self.birth_date = birth_date
        self.passport_number = passport_number
        self.issue_date = issue_date
        self.issued_by = issued_by

    def get_passport_data(self):
        return {
            "full_name": self.full_name,
            "birth_date": self.birth_date,
            "passport_number": self.passport_number,
            "issue_date": self.issue_date,
            "issued_by": self.issued_by,
        }

    def verify_passport(self):
        return True

    def display_passport(self):

        data = self.get_passport_data()
        return f"""
┌──────────────────── ПАСПОРТ РФ ────────────────────┐
│ ФИО: {data['full_name']:<35} │
│ Дата рождения: {data['birth_date']:<28} │
│ Номер паспорта: {data['passport_number']:<27} │
│ Дата выдачи: {data['issue_date']:<29} │
│ Кем выдан: {data['issued_by']:<30} │
└──────────────────────────────────────────────────┘
        """


class InsuranceSystem:

    def __init__(self, policy_number, company, valid_until):
        self.policy_number = policy_number
        self.company = company
        self.valid_until = valid_until

    def get_insurance_data(self):
        return {
            "policy_number": self.policy_number,
            "company": self.company,
            "valid_until": self.valid_until,
        }

    def check_coverage(self, service_type):
        return service_type in [
            "амбулаторная",
            "стационарная",
            "скорая помощь",
            "стоматология",
        ]
```

```
    def display_insurance(self):
        data = self.get_insurance_data()
        return f"""
┌──────────────────── МЕДИЦИНСКИЙ ПОЛИС ────────────────────┐
│ Номер полиса: {data['policy_number']:<30} │
│ Страховая компания: {data['company']:<25} │
│ Действителен до: {data['valid_until']:<27} │
└────────────────────────────────────────────────────────────┘
        """


class BankCardSystem:

    def __init__(self, card_number, owner_name, expiry_date, initial_balance):
        self.card_number = card_number
        self.owner_name = owner_name
        self.balance = initial_balance
        self.expiry_date = expiry_date
        self.transactions = []

    def get_card_data(self):
        return {
            "card_number": f"**** **** **** {self.card_number[-4:]}",
            "owner": self.owner_name,
            "balance": self.balance,
            "expiry": self.expiry_date,
        }

    def withdraw(self, amount):
        if self.balance >= amount:
            self.balance -= amount
            self.transactions.append(f"-{amount} руб.")
            return True, f"Снято {amount} руб. Остаток: {self.balance} руб."
        else:
            return False, f"Недостаточно средств. Доступно: {self.balance} руб."

    def deposit(self, amount):
        self.balance += amount
        self.transactions.append(f"+{amount} руб.")
        return f"Счет пополнен на {amount} руб. Баланс: {self.balance} руб."

    def display_card(self):
        data = self.get_card_data()
        return f"""
┌──────────────────── БАНКОВСКАЯ КАРТА ──────────────────────┐
│ Владелец: {data['owner']:<35} │
│ Номер карты: {data['card_number']:<30} │
│ Срок действия: {data['expiry']:<30} │
│ Баланс: {data['balance']:<20} руб.│
└────────────────────────────────────────────────────────────┘
        """


class UniversalCardInterface(ABC):

    @abstractmethod
    def identify(self):
        pass

    @abstractmethod
    def pay(self, amount):
        pass

    @abstractmethod
    def show_insurance(self):
        pass

    @abstractmethod
    def get_full_info(self):
```

```
        pass


class UniversalCard(UniversalCardInterface):

    def __init__(self, passport, insurance, bank_card):
        self._passport = passport
        self._insurance = insurance
        self._bank_card = bank_card
        self.card_id = f"УЭК-{hash(self) % 10000:04d}"

    def identify(self):
        print("\n🔍 ИДЕНТИФИКАЦИЯ ЛИЧНОСТИ:")
        passport_data = self._passport.get_passport_data()
        return f"Гражданин: {passport_data['full_name']}, Паспорт:
{passport_data['passport_number']}"

    def pay(self, amount):
        print(f"\n💳 ОПЕРАЦИЯ: ПЛАТЕЖ на сумму {amount} руб.")
        success, message = self._bank_card.withdraw(amount)
        if success:
            return f"✅ {message}"
        else:
            return f"❌ {message}"

    def show_insurance(self):
        print("\n🏥 МЕДИЦИНСКАЯ СТРАХОВКА:")
        return self._insurance.display_insurance()

    def get_full_info(self):
        info = f"""
╔═══════════════════════════════════════════════════╗
║           УНИВЕРСАЛЬНАЯ ЭЛЕКТРОННАЯ КАРТА          ╖
╟═══════════════════════════════════════════════════╢
╟ ID карты: {self.card_id:<48} ╢
╙═══════════════════════════════════════════════════╜
        """
        return (
            info
            + self._passport.display_passport()
            + self._insurance.display_insurance()
            + self._bank_card.display_card()
        )

    def top_up_balance(self, amount):

        print(f"\n💰 ПОПОЛНЕНИЕ БАЛАНСА: +{amount} руб.")
        return self._bank_card.deposit(amount)

    def check_insurance_coverage(self, service):
        if self._insurance.check_coverage(service):
            return f"✅ Услуга '{service}' покрывается страховкой"
        else:
            return f"❌ Услуга '{service}' НЕ покрывается страховкой"


def input_passport_data():
    """Ввод паспортных данных"""
    print("\n📋 ВВОД ПАСПОРТНЫХ ДАННЫХ:")
    full_name = input("Введите ФИО полностью: ")
    birth_date = input("Введите дату рождения (ДД.ММ.ГГГГ): ")
    passport_number = input("Введите номер паспорта (серия номер): ")
    issue_date = input("Введите дату выдачи (ДД.ММ.ГГГГ): ")
    issued_by = input("Введите кем выдан паспорт: ")

    return PassportSystem(full_name, birth_date, passport_number, issue_date, issued_by)
```

```python
def input_insurance_data():
    print("\n📋 ВВОД ДАННЫХ СТРАХОВОГО ПОЛИСА:")
    policy_number = input("Введите номер полиса: ")
    company = input("Введите страховую компанию: ")
    valid_until = input("Введите срок действия до (ДД.ММ.ГГГГ): ")

    return InsuranceSystem(policy_number, company, valid_until)


def input_bank_card_data():
    print("\n💳 ВВОД ДАННЫХ БАНКОВСКОЙ КАРТЫ:")
    card_number = input("Введите номер карты (16 цифр): ")
    owner_name = input("Введите имя владельца (латиницей): ")
    expiry_date = input("Введите срок действия (ММ/ГГ): ")

    while True:
        try:
            initial_balance = float(input("Введите начальный баланс (руб.): "))
            break
        except ValueError:
            print("Ошибка: введите число")

    return BankCardSystem(card_number, owner_name, expiry_date, initial_balance)


def main():
    """Главная функция для демонстрации работы"""

    print("=" * 80)
    print("ПРОГРАММА: УНИВЕРСАЛЬНАЯ ЭЛЕКТРОННАЯ КАРТА")
    print("=" * 80)
    print("\nДобро пожаловать в систему оформления универсальной карты!")

    # Ввод данных пользователем
    print("\n" + "=" * 80)
    print("ЭТАП 1: ЗАПОЛНЕНИЕ ДАННЫХ ДЛЯ КАРТЫ")
    print("=" * 80)

    passport = input_passport_data()
    insurance = input_insurance_data()
    bank_card = input_bank_card_data()

    my_card = UniversalCard(passport, insurance, bank_card)

    print("\n" + "=" * 80)
    print("✅ УНИВЕРСАЛЬНАЯ КАРТА УСПЕШНО СОЗДАНА!")
    print("=" * 80)
    print(my_card.get_full_info())

    while True:
        print("\n" + "=" * 50)
        print("ДОСТУПНЫЕ ОПЕРАЦИИ С КАРТОЙ:")
        print("1. Идентификация личности")
        print("2. Показать медицинскую страховку")
        print("3. Проверить покрытие страховки")
        print("4. Совершить платеж")
        print("5. Пополнить баланс карты")
        print("6. Показать полную информацию")
        print("0. Выход")

        choice = input("\nВыберите операцию (0-6): ")

        if choice == "1":
            print(my_card.identify())

        elif choice == "2":
            print(my_card.show_insurance())

        elif choice == "3":
```

```
            service = input("Введите тип медицинской услуги: ")
            print(my_card.check_insurance_coverage(service))

        elif choice == "4":
            try:
                amount = float(input("Введите сумму платежа: "))
                print(my_card.pay(amount))
            except ValueError:
                print("Ошибка: введите число")

        elif choice == "5":
            try:
                amount = float(input("Введите сумму пополнения: "))
                print(my_card.top_up_balance(amount))
            except ValueError:
                print("Ошибка: введите число")

        elif choice == "6":
            print(my_card.get_full_info())

        elif choice == "0":
            break

        else:
            print("Неверный выбор. Попробуйте снова.")

    print("\n" + "=" * 80)
    print("СПАСИБО ЗА ИСПОЛЬЗОВАНИЕ УНИВЕРСАЛЬНОЙ КАРТЫ!")
    print("=" * 80)


if __name__ == "__main__":
    main()
```

```
Выберите операцию (0-6): 1

🔍 ИДЕНТИФИКАЦИЯ ЛИЧНОСТИ:
Гражданин: матюх игорь , Паспорт: 432343
```

```
┌──────────────────── МЕДИЦИНСКИЙ ПОЛИС ────────────────────┐
│ Номер полиса: 4342315                                      │
│ Страховая компания: Атлас                                  │
│ Действителен до: 08.09.2010                                │
└────────────────────────────────────────────────────────────┘
```

```
Выберите операцию (0-6): 4
Введите сумму платежа: 2000000

💳 ОПЕРАЦИЯ: ПЛАТЕЖ на сумму 2000000.0 руб.
✅ Снято 2000000.0 руб. Остаток: 998000000.0 руб.
```

Задание 3: Проект «Принтеры». В проекте должны быть реализованы разные модели принтеров, которые выполняют разные виды печати.

Код программы:

```
from abc import ABC, abstractmethod
from datetime import datetime


class PrintStrategy(ABC):

    @abstractmethod
    def print_document(self, document_text, copies=1):
        pass

    @abstractmethod
    def get_printer_type(self):
        pass

    @abstractmethod
    def get_cost_per_page(self):
        pass


class InkjetPrintStrategy(PrintStrategy):

    def __init__(self):
        self.ink_level = 100
        self.name = "Струйный принтер"

    def print_document(self, document_text, copies=1):

        result = []
        pages = max(1, len(document_text) // 1000 + 1)

        print(f"\n🖨️ Идет струйная печать...")
        print(f"📊 Расход чернил: {pages * 0.5:.1f}% на копию")

        for copy in range(1, copies + 1):
            result.append(f"\n📄 Копия {copy}:")
            result.append("=" * 50)
            result.append(f"🖨️ Тип печати: {self.get_printer_type()}")
            result.append(f"⚙️ Качество: 1200 dpi (высокое)")
            result.append(f"📝 Текст документа:")
            result.append(f"   {document_text}")
            result.append(
                f"💰 Стоимость копии: {pages * self.get_cost_per_page():.2f} руб."
            )

            self.ink_level -= pages * 0.5

        return "\n".join(result)

    def get_printer_type(self):
        return f"{self.name} (чернила: {max(0, self.ink_level):.1f}%)"

    def get_cost_per_page(self):
        return 2.5
```

```python
class LaserPrintStrategy(PrintStrategy):

    def __init__(self):
        self.toner_level = 100
        self.name = "Лазерный принтер"

    def print_document(self, document_text, copies=1):
        result = []
        pages = max(1, len(document_text) // 1500 + 1)

        print(f"\n🖨️ Идет лазерная печать...")
        print(f"📊 Расход тонера: {pages * 0.3:.1f}% на копию")

        for copy in range(1, copies + 1):
            result.append(f"\n📄 Копия {copy}:")
            result.append("=" * 50)
            result.append(f"🖨️ Тип печати: {self.get_printer_type()}")
            result.append(f"⚙️ Качество: 600 dpi (быстрая)")
            result.append(f"📝 Текст документа:")
            result.append(f"   {document_text}")
            result.append(
                f"💰 Стоимость копии: {pages * self.get_cost_per_page():.2f} руб."
            )

            self.toner_level -= pages * 0.3

        return "\n".join(result)

    def get_printer_type(self):
        return f"{self.name} (тонер: {max(0, self.toner_level):.1f}%)"

    def get_cost_per_page(self):
        return 1.8


class DotMatrixPrintStrategy(PrintStrategy):

    def __init__(self):
        self.ribbon_level = 100
        self.name = "Матричный принтер"

    def print_document(self, document_text, copies=1):
        result = []
        pages = max(1, len(document_text) // 800 + 1)

        print(f"\n🖨️ Идет матричная печать (очень шумно)...")
        print(f"📊 Износ ленты: {pages * 0.8:.1f}% на копию")

        for copy in range(1, copies + 1):
            result.append(f"\n📄 Копия {copy}:")
            result.append("=" * 50)
            result.append(f"🖨️ Тип печати: {self.get_printer_type()}")
            result.append(f"⚙️ Режим: Черновик")
            result.append(f"📝 Текст документа:")
            result.append(f"   {document_text}")
            result.append(
                f"💰 Стоимость копии: {pages * self.get_cost_per_page():.2f} руб."
            )

            self.ribbon_level -= pages * 0.8

        return "\n".join(result)

    def get_printer_type(self):
        return f"{self.name} (лента: {max(0, self.ribbon_level):.1f}%)"
```

```
    def get_cost_per_page(self):
        return 0.9


class Printer:

    def __init__(self, model, strategy: PrintStrategy):
        self.model = model
        self._strategy = strategy
        self.print_count = 0
        self.last_maintenance = datetime.now()

    def set_strategy(self, strategy: PrintStrategy):
        old = self._strategy.get_printer_type()
        self._strategy = strategy
        print(f"\n🔄 Смена режима печати:")
        print(f"   Было: {old}")
        print(f"   Стало: {self._strategy.get_printer_type()}")

    def print_document(self, document_text, copies=1):
        self.print_count += copies

        print(f"\n{'=' * 60}")
        print(f"🖨️ ПРИНТЕР {self.model}")
        print(f"📊 Напечатано всего: {self.print_count} копий")
        print(f"{'=' * 60}")

        result = self._strategy.print_document(document_text, copies)
        print(f"\n✅ Печать завершена!")
        return result

    def get_info(self):
        return f"""
╔═══════════════════════════════════════════╗
   ИНФОРМАЦИЯ О ПРИНТЕРЕ
╠═══════════════════════════════════════════╣
 Модель: {self.model}
 Тип: {self._strategy.get_printer_type()}
 Напечатано: {self.print_count} копий
 Последнее обслуживание: {self.last_maintenance.strftime('%d.%m.%Y')}
╚═══════════════════════════════════════════╝
        """


def input_document():

    print("\n📝 ВВОД ДОКУМЕНТА:")
    print("(введите текст документа, для завершения введите 'end' на отдельной строке)")

    lines = []
    while True:
        line = input()
        if line == "end":
            break
        lines.append(line)

    return "\n".join(lines) if lines else input("Введите текст документа: ")


def get_printer_strategy():

    print("\n🖨️ ДОСТУПНЫЕ ТИПЫ ПРИНТЕРОВ:")
    print("1. Струйный принтер (для фотографий и цветной печати)")
    print("2. Лазерный принтер (для документов, быстро и дешево)")
    print("3. Матричный принтер (для накладных, шумно но надежно)")

    while True:
```

```
        try:
            choice = int(input("\nВыберите тип принтера (1-3): "))
            if choice == 1:
                return InkjetPrintStrategy()
            elif choice == 2:
                return LaserPrintStrategy()
            elif choice == 3:
                return DotMatrixPrintStrategy()
            else:
                print("Ошибка: введите число от 1 до 3")
        except ValueError:
            print("Ошибка: введите число")


def main():

    print("=" * 80)
    print("ПРОГРАММА: РАЗНЫЕ МОДЕЛИ ПРИНТЕРОВ")
    print("=" * 80)

    print("\n🔧 СОЗДАНИЕ ПРИНТЕРА")
    model = input("Введите модель принтера: ")
    strategy = get_printer_strategy()

    printer = Printer(model, strategy)
    print(f"\n✅ Принтер {model} создан!")
    print(printer.get_info())

    while True:
        print("\n" + "=" * 50)
        print("ДОСТУПНЫЕ ОПЕРАЦИИ:")
        print("1. Напечатать документ")
        print("2. Сменить тип принтера")
        print("3. Показать информацию о принтере")
        print("0. Выход")

        choice = input("\nВыберите операцию (0-3): ")

        if choice == "1":

            doc_text = input_document()

            try:
                copies = int(input("Введите количество копий: "))
                if copies < 1:
                    copies = 1
            except ValueError:
                print("Использовано значение по умолчанию: 1")
                copies = 1

            result = printer.print_document(doc_text, copies)
            print(result)

        elif choice == "2":

            new_strategy = get_printer_strategy()
            printer.set_strategy(new_strategy)

        elif choice == "3":
            print(printer.get_info())

        elif choice == "0":
            break

        else:
            print("Неверный выбор. Попробуйте снова.")

    print("\n" + "=" * 80)
    print(f"СПАСИБО ЗА ИСПОЛЬЗОВАНИЕ ПРИНТЕРА {model}!")
```

```
    print("=" * 80)


if __name__ == "__main__":
    main()
```

```
📄 Копия 1:
═══════════════════════════════════════════════════════════════════════════
🖨️ Тип печати: Струйный принтер (чернила: 100.0%)
⚙️ Качество: 1200 dpi (высокое)
📝 Текст документа:
   ПРИВЕТ Я АВДЕЙ
💰 Стоимость копии: 2.50 руб.
```

Вывод: приобрел навыки применения паттернов проектирования при решении практических задач с использованием языка Python.